Evidencias de las pruebas

Generadas el 2026-11-14 sobre la rama `dev/chris`, commit `bb03b8c`. Se usó el stack de Docker Compose (MariaDB, Broker y Sistema 1, cada uno en su propio proceso y contenedor) y clientes de evidencias que **solo conocen la dirección del Broker** (`localhost:5000`). Los clientes usan las clases `Solicitud`, `Respuesta` y `GsonFactory` del módulo `common`, igual que el Sistema 2.

Cada prueba se presenta en su propia sección con el log que generó: el JSON enviado, el JSON recibido y las líneas de la **bitácora del Broker** (`docker logs uady-broker`) que produjo esa solicitud.

Pruebas requeridas por la actividad

#	Prueba requerida	Evidencia	Resultado
1	Registro correcto de proveedores	`01_registro_proveedores.txt`	Al arrancar, el Sistema 1 registra sus 3 servicios (`gestorTiendas`, `inventario`, `gestorPrecios`) en el Broker; la bitácora muestra cada `[REGISTRO] registrar … -> sistema-centralizado:6000`
2	Localización de un servicio	`02_localizacion_servicio.txt`	`inventario.listarInventario` → el Broker localiza `sistema-centralizado:6000` → `EXITO`
3			`gestorTiendas.listarTiendas`,

#	Prueba requerida	Evidencia	Resultado
	Ejecución de una operación (una por servicio)	`03_ejecucion_operacion.txt`	`inventario.listarInventario` y `gestorPrecios.listarPreciosProducto` → `EXITO`
4	Respuesta exitosa	`04_respuesta_exitosa.txt`	Venta de `SN-UADY-0003` → `EXITO` con id, resultado, estado y fecha y hora; la unidad queda `VENDIDO` en la BD
5	Servicio desconocido	`05_servicio_desconocido.txt`	`servicioFantasma` → `ERROR SERVICIO_DESCONOCIDO`, sin contactar a ningún proveedor
6	Parámetro inválido	`06_parametro_invalido.txt`	Falta `numero_serie`, `id_tienda` negativo y producto inexistente → `PARAMETROS_INVALIDOS` con mensaje claro
7	Excepción producida por un proveedor	`07_excepcion_proveedor.txt`	Con la tabla `tiendas_almacenes` ausente, el proveedor lanza `SQLException` → `ERROR_INTERNO`; el Broker sigue atendiendo y, al restaurarla, la

#	Prueba requerida	Evidencia	Resultado
			operación funciona sin reiniciar nada. Caso B: con MariaDB detenida, el Broker responde `TIMEOUT` tras 3 intentos
8	Cambio de proveedor sin modificar el cliente	`08_cambio_proveedor.txt`	El mismo comando del cliente lo atiende primero `sistema-centralizado:6000` y, tras registrarse un segundo proveedor y apagar el original, `host.docker.internal:6001`

Requisitos del Nivel 2 y preguntas de análisis

#	Requisito / pregunta	Evidencia	Resultado
9	Timeouts, reintentos limitados y desconexiones	`09_proveedor_caido_timeout.txt`	Proveedor apagado: 3 intentos por solicitud (`intento 1/3 … 3/3`), tras 3 solicitudes fallidas el servicio queda `NO DISPONIBLE` y se rechaza al instante; al volver el proveedor se reactiva
10	Pruebas automatizadas	`10_pruebas_automatizadas.txt`	`mvn test` con MariaDB: **195 pruebas, 0 fallos,**

#	Requisito / pregunta	Evidencia	Resultado
			0 omitidas (Common 10, Broker 47, Sistema 1 129, Sistema 2 9)
11	Atención concurrente con hilos	`11_concurrencia_hilos.txt`	20 clientes simultáneos: **20/20** reciben `EXITO` con el id de su propia solicitud; los atendieron **16 hilos distintos** del pool
12	Pregunta 3 y 4: el Broker deja de funcionar / punto único de falla	`12_broker_caido.txt`	Sin Broker, el cliente no llega a ningún servicio aunque el Sistema 1 siga vivo. Al reiniciar el Broker, su registro en memoria está vacío (`SERVICIO_DESCONOCIDO`) hasta que el Sistema 1 se vuelve a registrar
13	Pregunta 9: solicitudes duplicadas	`13_solicitud_duplicada.txt`	La misma venta dos veces: la segunda se rechaza (`ya fue vendida`) y en la BD queda **una sola** venta
14	Pregunta 7: costo en rendimiento	`14_costo_rendimiento.txt`	100 solicitudes por camino: directo 3.53 ms, por el Broker 4.45 ms de promedio →

#	Requisito / pregunta	Evidencia	Resultado
			+0.92 ms por solicitud
	Bitácora de solicitudes / mensajes en consola / pregunta 10 (monitoreo)	Todas	Cada archivo incluye las líneas `[BROKER]` y `[REGISTRO]` de la bitácora

Prueba 1: Registro correcto de proveedores

Pruebas requeridas por la actividad

Evidencia: `01_registro_proveedores.txt`

Resultado: Al arrancar, el Sistema 1 registra sus 3 servicios (`gestorTiendas` , `inventario` , `gestorPrecios`) en el Broker; la bitácora muestra cada `[REGISTRO] registrar … -> sistema-centralizado:6000`

```
# Prueba 1: Registro correcto de proveedores
Fecha (hora local del equipo): 2026-11-14 09:18:12
Codigo: rama dev/chris, commit bb03b8c

Al arrancar, el Sistema 1 (proveedor) se registra dinamicamente en el
Broker enviando la operacion
broker.registrarProveedor por socket, una vez por cada servicio que ofrece.
El Broker asocia cada
nombre de servicio con el host y puerto del proveedor.

## Consola del Sistema 1 (docker logs uady-sistema-centralizado)
  Escuchando en el puerto 6000 con pool de 16 hilos
  Servicios: gestorTiendas, inventario, gestorPrecios
Proveedor registrado en el broker broker:5000 como sistema-
centralizado:6000 (gestorTiendas, inventario, gestorPrecios)

## Bitacora del Broker (docker logs uady-broker)
Broker iniciado en el puerto 5000 con pool de 16 hilos (timeout conexion
2000 ms, timeout lectura 5000 ms, reintentos 2, fallos para no disponible
3, enfriamiento 30000 ms)
[REGISTRO] 2026-11-14 15:18:04.923 [pool-1-thread-2] registrar
gestorTiendas -> sistema-centralizado:6000
[BROKER] 2026-11-14 15:18:04.927 [pool-1-thread-2]
id=4a9115a3-12a4-487c-94d3-fcd6bbb9ea57 broker.registrarProveedor -> broker
| EXITO | 64 ms
[REGISTRO] 2026-11-14 15:18:04.946 [pool-1-thread-3] registrar inventario
-> sistema-centralizado:6000
[BROKER] 2026-11-14 15:18:04.946 [pool-1-thread-3] id=98ab0ce6-4ee9-4fa0-
aaad-4e0876f6904f broker.registrarProveedor -> broker | EXITO | 0 ms
[REGISTRO] 2026-11-14 15:18:04.948 [pool-1-thread-4] registrar
gestorPrecios -> sistema-centralizado:6000
[BROKER] 2026-11-14 15:18:04.948 [pool-1-thread-4] id=59e22cf3-
e34b-422e-95fa-d811eec592d5 broker.registrarProveedor -> broker | EXITO | 0
```

```
ms

Resumen: los 3 servicios (gestorTiendas, inventario, gestorPrecios) quedan
registrados hacia
sistema-centralizado:6000. Prueba automatizada relacionada:
RegistroServiciosTest (registrarYBuscar).
```

Prueba 2: Localización de un servicio

Pruebas requeridas por la actividad

Evidencia: `02_localizacion_servicio.txt`

Resultado: `inventario.listarInventario` → el Broker localiza `sistema-centralizado:6000` → `EXITO`

```
# Prueba 2: Localizacion de un servicio
Fecha (hora local del equipo): 2026-11-14 09:18:12
Codigo: rama dev/chris, commit bb03b8c

El cliente solo conoce la direccion del Broker (localhost:5000). El Broker
busca en su registro el
proveedor del servicio 'inventario' y le reenvia la solicitud. La bitacora
muestra el proveedor localizado.

## inventario.listarInventario

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-02",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:13.3713633"
}
BROKER -> CLIENTE (105 ms)
{
  "idSolicitud": "ev-02",
  "resultado": [
    {
      "numeroSerie": "SN-UADY-0001",
      "idProducto": 1,
      "nombreProducto": "Laptop Lenovo ThinkPad E14",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0003",
      "idProducto": 2,
      "nombreProducto": "Proyector Epson PowerLite",
      "estado": "EN_ALMACEN"
```

```
    },
    {
      "numeroSerie": "SN-UADY-0005",
      "idProducto": 3,
      "nombreProducto": "Silla ergonomica",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0006",
      "idProducto": 3,
      "nombreProducto": "Silla ergonomica",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0007",
      "idProducto": 4,
      "nombreProducto": "Impresora laser HP",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0009",
      "idProducto": 5,
      "nombreProducto": "Tablet Samsung Galaxy Tab",
      "estado": "EN_ALMACEN"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:18:13.49528742"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:13.496 [pool-1-thread-6] id=ev-02
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 90 ms
```

Prueba 3: Ejecución de una operación (una por servicio)

Pruebas requeridas por la actividad

Evidencia: `03_ejecucion_operacion.txt`

Resultado: `gestorTiendas.listarTiendas`, `inventario.listarInventario` y `gestorPrecios.listarPreciosProducto` → `EXITO`


```
# Prueba 3: Ejecucion de una operacion (una por cada servicio registrado)
Fecha (hora local del equipo): 2026-11-14 09:18:13
Codigo: rama dev/chris, commit bb03b8c

Se ejecuta una operacion de cada uno de los 3 servicios registrados. Todas
viajan por el Broker
como JSON por socket y llegan al proveedor localizado.

## Servicio gestorTiendas: listarTiendas

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-03a",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "listarTiendas",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:14.0694684"
}
BROKER -> CLIENTE (26 ms)
{
  "idSolicitud": "ev-03a",
  "resultado": [
    {
      "idTienda": 1,
      "nombre": "Almacen Central UADY",
      "direccion": "Calle 60 s/n, Centro, Merida, Yucatan"
    },
    {
      "idTienda": 2,
      "nombre": "Tienda Facultad de Ingenieria",
      "direccion": "Periferico Norte, Campus de Ingenieria, Merida"
    },
    {
      "idTienda": 3,
```

```
      "nombre": "Tienda Facultad de Matematicas",
      "direccion": "Campus de Ciencias Exactas, Merida"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:18:14.114145834"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:14.115 [pool-1-thread-7] id=ev-03a
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 11 ms

## Servicio inventario: listarInventario

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-03b",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:14.6735004"
}
BROKER -> CLIENTE (22 ms)
{
  "idSolicitud": "ev-03b",
  "resultado": [
    {
      "numeroSerie": "SN-UADY-0001",
      "idProducto": 1,
      "nombreProducto": "Laptop Lenovo ThinkPad E14",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0003",
      "idProducto": 2,
      "nombreProducto": "Proyector Epson PowerLite",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0005",
      "idProducto": 3,
      "nombreProducto": "Silla ergonomica",
      "estado": "EN_ALMACEN"
    },
```

```
    {
      "numeroSerie": "SN-UADY-0006",
      "idProducto": 3,
      "nombreProducto": "Silla ergonomica",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0007",
      "idProducto": 4,
      "nombreProducto": "Impresora laser HP",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0009",
      "idProducto": 5,
      "nombreProducto": "Tablet Samsung Galaxy Tab",
      "estado": "EN_ALMACEN"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:18:14.714569046"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:14.715 [pool-1-thread-9] id=ev-03b
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 8 ms

## Servicio gestorPrecios: listarPreciosProducto

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-03c",
  "nombreServicio": "gestorPrecios",
  "nombreOperacion": "listarPreciosProducto",
  "parametros": {
    "id_producto": 1
  },
  "fechaHora": "2026-11-14T09:18:15.2262593"
}
BROKER -> CLIENTE (39 ms)
{
  "idSolicitud": "ev-03c",
  "resultado": {
    "idProducto": 1,
```

```
    "nombre": "Laptop Lenovo ThinkPad E14",
    "precioActual": 18999.00,
    "historial": []
  },
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:18:15.283064142"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:15.283 [pool-1-thread-10] id=ev-03c
gestorPrecios.listarPreciosProducto -> sistema-centralizado:6000 | EXITO |
25 ms
```

Prueba 4: Respuesta exitosa

Pruebas requeridas por la actividad

Evidencia: `04_respuesta_exitosa.txt`

Resultado: Venta de `SN-UADY-0003` → `EXITO` con id, resultado, estado y fecha y hora; la unidad queda `VENDIDO` en la BD

```
# Prueba 4: Respuesta exitosa
Fecha (hora local del equipo): 2026-11-14 09:18:15
Codigo: rama dev/chris, commit bb03b8c

El cliente registra la venta de una unidad (CU2). El Sistema 1 la procesa
en una transaccion y la
respuesta incluye id de la solicitud, resultado, estado y fecha y hora.

## gestorTiendas.actualizarTiendaVenta

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-04",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "actualizarTiendaVenta",
  "parametros": {
    "numero_serie": "SN-UADY-0003",
    "id_tienda": 1,
    "id_usuario": 2
  },
  "fechaHora": "2026-11-14T09:18:15.8278687"
}
BROKER -> CLIENTE (24 ms)
{
  "idSolicitud": "ev-04",
  "resultado": {
    "numero_serie": "SN-UADY-0003",
    "id_tienda": 1,
    "fecha_venta": "2026-11-14T15:18:15"
  },
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:18:15.875516306"
}
Resumen: estado=EXITO
```

```
Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:15.876 [pool-1-thread-11] id=ev-04
gestorTiendas.actualizarTiendaVenta -> sistema-centralizado:6000 | EXITO |
8 ms

Estado en la base de datos:
numero_serie    estado  id_tienda   id_usuario  fecha_venta
SN-UADY-0003    VENDIDO 1   2   2026-11-14 15:18:15
```

Prueba 5: Servicio desconocido

Pruebas requeridas por la actividad

Evidencia: `05_servicio_desconocido.txt`

Resultado: `servicioFantasma` → `ERROR SERVICIO_DESCONOCIDO`, sin contactar a ningún proveedor

```
# Prueba 5: Servicio desconocido
Fecha (hora local del equipo): 2026-11-14 09:18:16
Codigo: rama dev/chris, commit bb03b8c

El cliente pide un servicio que ningun proveedor registro. El Broker lo
rechaza sin contactar a
ningun proveedor y sin caerse.

## servicioFantasma.listar

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-05",
  "nombreServicio": "servicioFantasma",
  "nombreOperacion": "listar",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:16.6061441"
}
BROKER -> CLIENTE (14 ms)
{
  "idSolicitud": "ev-05",
  "estado": "ERROR",
  "codigoError": "SERVICIO_DESCONOCIDO",
  "mensajeError": "Servicio desconocido: servicioFantasma",
  "fechaHora": "2026-11-14T15:18:16.638050827"
}
Resumen: estado=ERROR, codigo=SERVICIO_DESCONOCIDO, mensaje="Servicio
desconocido: servicioFantasma"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:16.638 [pool-1-thread-12] id=ev-05
servicioFantasma.listar -> (sin proveedor) | ERROR SERVICIO_DESCONOCIDO:
Servicio desconocido: servicioFantasma | 0 ms
```

Prueba 6: Parámetro inválido

Pruebas requeridas por la actividad

Evidencia: `06_parametro_invalido.txt`

Resultado: Falta `numero_serie`, `id_tienda` negativo y producto inexistente → `PARAMETROS_INVALIDOS` con mensaje claro

```
# Prueba 6: Parametro invalido
Fecha (hora local del equipo): 2026-11-14 09:18:16
Codigo: rama dev/chris, commit bb03b8c

Se envian ventas con parametros faltantes o invalidos. El proveedor los
valida y responde
PARAMETROS_INVALIDOS con un mensaje claro, conservando el id de la
solicitud.

## Falta numero_serie

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-06a",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "actualizarTiendaVenta",
  "parametros": {
    "id_tienda": 1,
    "id_usuario": 2
  },
  "fechaHora": "2026-11-14T09:18:17.1976449"
}
BROKER -> CLIENTE (17 ms)
{
  "idSolicitud": "ev-06a",
  "estado": "ERROR",
  "codigoError": "PARAMETROS_INVALIDOS",
  "mensajeError": "Falta el parámetro numero_serie",
  "fechaHora": "2026-11-14T15:18:17.234579654"
}
Resumen: estado=ERROR, codigo=PARAMETROS_INVALIDOS, mensaje="Falta el
parámetro numero_serie"

Bitacora del Broker (docker logs uady-broker):
```

```
[BROKER] 2026-11-14 15:18:17.235 [pool-1-thread-13] id=ev-06a
gestorTiendas.actualizarTiendaVenta -> sistema-centralizado:6000 | ERROR
PARAMETROS_INVALIDOS: Falta el parámetro numero_serie | 2 ms

## id_tienda negativo

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-06b",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "actualizarTiendaVenta",
  "parametros": {
    "numero_serie": "SN-UADY-0001",
    "id_tienda": -5,
    "id_usuario": 2
  },
  "fechaHora": "2026-11-14T09:18:17.7366356"
}
BROKER -> CLIENTE (16 ms)
{
  "idSolicitud": "ev-06b",
  "estado": "ERROR",
  "codigoError": "PARAMETROS_INVALIDOS",
  "mensajeError": "El parámetro id_tienda debe ser positivo",
  "fechaHora": "2026-11-14T15:18:17.774421699"
}
Resumen: estado=ERROR, codigo=PARAMETROS_INVALIDOS, mensaje="El parámetro
id_tienda debe ser positivo"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:17.774 [pool-1-thread-14] id=ev-06b
gestorTiendas.actualizarTiendaVenta -> sistema-centralizado:6000 | ERROR
PARAMETROS_INVALIDOS: El parámetro id_tienda debe ser positivo | 2 ms

## Producto inexistente en gestorPrecios

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-06c",
  "nombreServicio": "gestorPrecios",
  "nombreOperacion": "listarPreciosProducto",
  "parametros": {
    "id_producto": 999
  },
  "fechaHora": "2026-11-14T09:18:18.327309"
```

```
}
BROKER -> CLIENTE (22 ms)
{
  "idSolicitud": "ev-06c",
  "estado": "ERROR",
  "codigoError": "PARAMETROS_INVALIDOS",
  "mensajeError": "El producto 999 no existe",
  "fechaHora": "2026-11-14T15:18:18.369878855"
}
Resumen: estado=ERROR, codigo=PARAMETROS_INVALIDOS, mensaje="El producto
999 no existe"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:18.370 [pool-1-thread-15] id=ev-06c
gestorPrecios.listarPreciosProducto -> sistema-centralizado:6000 | ERROR
PARAMETROS_INVALIDOS: El producto 999 no existe | 6 ms
```

Prueba 7: Excepción producida por un proveedor

Pruebas requeridas por la actividad

Evidencia: `07_excepcion_proveedor.txt`

Resultado: Con la tabla `tiendas_almacenes` ausente, el proveedor lanza `SQLException` → `ERROR_INTERNO`; el Broker sigue atendiendo y, al restaurarla, la operación funciona sin reiniciar nada. Caso B: con MariaDB detenida, el Broker responde `TIMEOUT` tras 3 intentos

```
# Prueba 7: Excepcion producida por un proveedor
Fecha (hora local del equipo): 2026-11-14 09:20:41
Codigo: rama dev/chris, commit bb03b8c

Caso A: se provoca una excepcion real dentro del proveedor (la tabla
'tiendas_almacenes' deja de existir de forma
temporal, asi que la consulta JDBC lanza SQLException). El despachador del
Sistema 1 convierte la
excepcion en una respuesta ERROR_INTERNO, conservando el id; el Broker la
devuelve al cliente y sigue
atendiendo. Al restaurar la tabla, la misma operacion funciona sin
reiniciar el Broker ni el Sistema 1.

Caso B (complemento): si la base de datos se detiene por completo, el
proveedor no responde dentro del
timeout de lectura del Broker (5 s); el Broker reintenta 3 veces y responde
TIMEOUT.

## Caso A, paso 1: RENAME TABLE tiendas_almacenes TO tiendas_respaldo

## Caso A, paso 2: solicitud mientras la tabla no existe

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-07-c1",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "listarTiendas",
  "parametros": {},
  "fechaHora": "2026-11-14T09:20:42.0394307"
}
BROKER -> CLIENTE (19 ms)
```

```
{
  "idSolicitud": "ev-07-c1",
  "estado": "ERROR",
  "codigoError": "ERROR_INTERNO",
  "mensajeError": "Error al ejecutar la operacion listarTiendas: No se pudo
consultar las tiendas",
  "fechaHora": "2026-11-14T15:20:42.080603877"
}
Resumen: estado=ERROR, codigo=ERROR_INTERNO, mensaje="Error al ejecutar la
operacion listarTiendas: No se pudo consultar las tiendas"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:20:42.081 [pool-1-thread-15] id=ev-07-c1
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | ERROR
ERROR_INTERNO: Error al ejecutar la operacion listarTiendas: No se pudo
consultar las tiendas | 5 ms

Consola del Sistema 1 en ese momento (docker logs uady-sistema-
centralizado):
Proveedor registrado en el broker broker:5000 como sistema-
centralizado:6000 (gestorTiendas, inventario, gestorPrecios)
  Servicios: gestorTiendas, inventario, gestorPrecios
Proveedor registrado en el broker broker:5000 como sistema-
centralizado:6000 (gestorTiendas, inventario, gestorPrecios)

## Caso A, paso 3: el Broker sigue atendiendo otras operaciones

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-07-c2",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:20:42.70822"
}
BROKER -> CLIENTE (22 ms)
{
  "idSolicitud": "ev-07-c2",
  "resultado": [
    {
      "numeroSerie": "SN-UADY-0001",
      "idProducto": 1,
      "nombreProducto": "Laptop Lenovo ThinkPad E14",
      "estado": "EN_ALMACEN"
    },
```

```
    {
      "numeroSerie": "SN-UADY-0006",
      "idProducto": 3,
      "nombreProducto": "Silla ergonomica",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0007",
      "idProducto": 4,
      "nombreProducto": "Impresora laser HP",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0009",
      "idProducto": 5,
      "nombreProducto": "Tablet Samsung Galaxy Tab",
      "estado": "EN_ALMACEN"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:20:42.747698035"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:20:42.748 [pool-1-thread-16] id=ev-07-c2
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 7 ms

## Caso A, paso 4: RENAME TABLE tiendas_respaldo TO tiendas_almacenes

## Caso A, paso 5: la misma solicitud, sin reiniciar nada

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-07-c3",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "listarTiendas",
  "parametros": {},
  "fechaHora": "2026-11-14T09:20:43.4464432"
}
BROKER -> CLIENTE (21 ms)
{
  "idSolicitud": "ev-07-c3",
  "resultado": [
    {
```

```
      "idTienda": 1,
      "nombre": "Almacen Central UADY",
      "direccion": "Calle 60 s/n, Centro, Merida, Yucatan"
    },
    {
      "idTienda": 2,
      "nombre": "Tienda Facultad de Ingenieria",
      "direccion": "Periferico Norte, Campus de Ingenieria, Merida"
    },
    {
      "idTienda": 3,
      "nombre": "Tienda Facultad de Matematicas",
      "direccion": "Campus de Ciencias Exactas, Merida"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:20:43.488581257"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:20:43.489 [pool-1-thread-1] id=ev-07-c3
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 5 ms

## Caso B, paso 1: docker stop uady-mariadb
## Caso B, paso 2: solicitud con la base detenida (cliente con espera de 30
s para ver la respuesta del Broker)

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-07-d1",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:20:44.4460508"
}
BROKER -> CLIENTE (15432 ms)
{
  "idSolicitud": "ev-07-d1",
  "estado": "ERROR",
  "codigoError": "TIMEOUT",
  "mensajeError": "Tiempo de espera agotado al contactar al proveedor",
  "fechaHora": "2026-11-14T15:20:59.898989283"
}
Resumen: estado=ERROR, codigo=TIMEOUT, mensaje="Tiempo de espera agotado al
```

```
contactar al proveedor"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:20:49.485 [pool-1-thread-2] id=ev-07-d1
inventario.listarInventario -> sistema-centralizado:6000 | intento 1/3
fallo: TIMEOUT (Tiempo de espera agotado al contactar al proveedor)
[BROKER] 2026-11-14 15:20:54.692 [pool-1-thread-2] id=ev-07-d1
inventario.listarInventario -> sistema-centralizado:6000 | intento 2/3
fallo: TIMEOUT (Tiempo de espera agotado al contactar al proveedor)
[BROKER] 2026-11-14 15:20:59.898 [pool-1-thread-2] id=ev-07-d1
inventario.listarInventario -> sistema-centralizado:6000 | intento 3/3
fallo: TIMEOUT (Tiempo de espera agotado al contactar al proveedor)
[BROKER] 2026-11-14 15:20:59.899 [pool-1-thread-2] id=ev-07-d1
inventario.listarInventario -> sistema-centralizado:6000 | ERROR TIMEOUT:
Tiempo de espera agotado al contactar al proveedor | 15419 ms

Observacion: el Broker tarda unos 15 s (3 intentos x 5 s) en responder
TIMEOUT. El cliente del
Sistema 2 espera 5 s, asi que el usuario veria primero su propio timeout.
Es la limitacion #1 de la
revision de errores (alinear el timeout del cliente con el peor caso del
Broker y no reintentar
operaciones que ya llegaron al proveedor).

## Caso B, paso 3: docker start uady-mariadb
## Caso B, paso 4: con la base de vuelta, sin reiniciar nada

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-07-d2",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:21:10.9237249"
}
BROKER -> CLIENTE (23 ms)
{
  "idSolicitud": "ev-07-d2",
  "resultado": [
    {
      "numeroSerie": "SN-UADY-0001",
      "idProducto": 1,
      "nombreProducto": "Laptop Lenovo ThinkPad E14",
      "estado": "EN_ALMACEN"
    },
```

```
    {
      "numeroSerie": "SN-UADY-0006",
      "idProducto": 3,
      "nombreProducto": "Silla ergonomica",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0007",
      "idProducto": 4,
      "nombreProducto": "Impresora laser HP",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0009",
      "idProducto": 5,
      "nombreProducto": "Tablet Samsung Galaxy Tab",
      "estado": "EN_ALMACEN"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:21:10.967079159"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:21:10.967 [pool-1-thread-5] id=ev-07-d2
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 7 ms
```

Prueba 8: Cambio de proveedor sin modificar el cliente

Pruebas requeridas por la actividad

Evidencia: `08_cambio_proveedor.txt`

Resultado: El mismo comando del cliente lo atiende primero `sistema-centralizado:6000` y, tras registrarse un segundo proveedor y apagar el original, `host.docker.internal:6001`


```
# Prueba 8: Cambio de proveedor sin modificar el cliente
Fecha (hora local del equipo): 2026-11-14 09:18:49
Codigo: rama dev/chris, commit bb03b8c

Un segundo proveedor (otra instancia del Sistema 1, fuera de Docker, en el
puerto 6001) se registra
en el Broker para los mismos servicios y sustituye al original; despues se
apaga el original. El cliente
ejecuta exactamente el mismo comando antes y despues: solo cambia el
proveedor que el Broker localiza.

## Paso 1: proveedor original

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-08a",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "listarTiendas",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:50.1432026"
}
BROKER -> CLIENTE (17 ms)
{
  "idSolicitud": "ev-08a",
  "resultado": [
    {
      "idTienda": 1,
      "nombre": "Almacen Central UADY",
      "direccion": "Calle 60 s/n, Centro, Merida, Yucatan"
    },
    {
      "idTienda": 2,
      "nombre": "Tienda Facultad de Ingenieria",
```

```
      "direccion": "Periferico Norte, Campus de Ingenieria, Merida"
    },
    {
      "idTienda": 3,
      "nombre": "Tienda Facultad de Matematicas",
      "direccion": "Campus de Ciencias Exactas, Merida"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:18:50.182351758"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:50.182 [pool-1-thread-10] id=ev-08a
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 3 ms

## Paso 2: arranca el proveedor 2 (host.docker.internal:6001) y se registra
Proveedor registrado en el broker localhost:5000 como
host.docker.internal:6001 (gestorTiendas, inventario, gestorPrecios)
[REGISTRO] 2026-11-14 15:18:50.503 [pool-1-thread-12] registrar
gestorTiendas -> host.docker.internal:6001
[REGISTRO] 2026-11-14 15:18:50.512 [pool-1-thread-14] registrar inventario
-> host.docker.internal:6001
[REGISTRO] 2026-11-14 15:18:50.514 [pool-1-thread-9] registrar
gestorPrecios -> host.docker.internal:6001

## Paso 3: se apaga el proveedor original (docker stop uady-sistema-
centralizado)

## Paso 4: el mismo cliente, sin cambios, ahora lo atiende el proveedor 2

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-08b",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "listarTiendas",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:52.5416426"
}
BROKER -> CLIENTE (78 ms)
{
  "idSolicitud": "ev-08b",
  "resultado": [
    {
```

```
      "idTienda": 1,
      "nombre": "Almacen Central UADY",
      "direccion": "Calle 60 s/n, Centro, Merida, Yucatan"
    },
    {
      "idTienda": 2,
      "nombre": "Tienda Facultad de Ingenieria",
      "direccion": "Periferico Norte, Campus de Ingenieria, Merida"
    },
    {
      "idTienda": 3,
      "nombre": "Tienda Facultad de Matematicas",
      "direccion": "Campus de Ciencias Exactas, Merida"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T09:18:52.638081"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:52.639 [pool-1-thread-3] id=ev-08b
gestorTiendas.listarTiendas -> host.docker.internal:6001 | EXITO | 64 ms
```

Prueba 9: Timeouts, reintentos limitados y desconexiones

Requisitos del Nivel 2 y preguntas de análisis

Evidencia: `09_proveedor_caido_timeout.txt`

Resultado: Proveedor apagado: 3 intentos por solicitud (`intento 1/3 … 3/3`), tras 3 solicitudes fallidas el servicio queda `NO DISPONIBLE` y se rechaza al instante; al volver el proveedor se reactiva

```
# Evidencia adicional: proveedor caido, reintentos limitados y desconexion
Fecha (hora local del equipo): 2026-11-14 09:18:52
Codigo: rama dev/chris, commit bb03b8c

Se apaga el proveedor activo (el proveedor 2) sin que se de de baja. El
Broker reintenta de forma
limitada (3 intentos, 200 ms entre ellos), responde con error y, tras 3
solicitudes fallidas, marca el
servicio como NO DISPONIBLE: la siguiente solicitud se rechaza de inmediato
sin esperar. Al volver un
proveedor y registrarse, el servicio se reactiva.

## Paso 1: se detiene el proveedor 2 (puerto 6001)

## Paso 2: primera solicitud (3 intentos)

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-09a",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:54.0235668"
}
BROKER -> CLIENTE (420 ms)
{
  "idSolicitud": "ev-09a",
  "estado": "ERROR",
  "codigoError": "ERROR_INTERNO",
  "mensajeError": "Error de comunicacion con el proveedor: Connection
refused (connect failed)",
  "fechaHora": "2026-11-14T15:18:54.469871499"
}
```

```
Resumen: estado=ERROR, codigo=ERROR_INTERNO, mensaje="Error de comunicacion
con el proveedor: Connection refused (connect failed)"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:54.065 [pool-1-thread-4] id=ev-09a
inventario.listarInventario -> host.docker.internal:6001 | intento 1/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:54.267 [pool-1-thread-4] id=ev-09a
inventario.listarInventario -> host.docker.internal:6001 | intento 2/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:54.469 [pool-1-thread-4] id=ev-09a
inventario.listarInventario -> host.docker.internal:6001 | intento 3/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:54.469 [pool-1-thread-4] id=ev-09a
inventario.listarInventario -> host.docker.internal:6001 | ERROR
ERROR_INTERNO: Error de comunicacion con el proveedor: Connection refused
(connect failed) | 406 ms

## Paso 3: segunda solicitud

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-09b",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:54.9872288"
}
BROKER -> CLIENTE (419 ms)
{
  "idSolicitud": "ev-09b",
  "estado": "ERROR",
  "codigoError": "ERROR_INTERNO",
  "mensajeError": "Error de comunicacion con el proveedor: Connection
refused (connect failed)",
  "fechaHora": "2026-11-14T15:18:55.425588035"
}
Resumen: estado=ERROR, codigo=ERROR_INTERNO, mensaje="Error de comunicacion
con el proveedor: Connection refused (connect failed)"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:55.021 [pool-1-thread-5] id=ev-09b
```

```
inventario.listarInventario -> host.docker.internal:6001 | intento 1/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:55.223 [pool-1-thread-5] id=ev-09b
inventario.listarInventario -> host.docker.internal:6001 | intento 2/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:55.425 [pool-1-thread-5] id=ev-09b
inventario.listarInventario -> host.docker.internal:6001 | intento 3/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:55.425 [pool-1-thread-5] id=ev-09b
inventario.listarInventario -> host.docker.internal:6001 | ERROR
ERROR_INTERNO: Error de comunicacion con el proveedor: Connection refused
(connect failed) | 405 ms

## Paso 4: tercera solicitud (alcanza el umbral)

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-09c",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:56.2072198"
}
BROKER -> CLIENTE (419 ms)
{
  "idSolicitud": "ev-09c",
  "estado": "ERROR",
  "codigoError": "ERROR_INTERNO",
  "mensajeError": "Error de comunicacion con el proveedor: Connection
refused (connect failed)",
  "fechaHora": "2026-11-14T15:18:56.645846663"
}
Resumen: estado=ERROR, codigo=ERROR_INTERNO, mensaje="Error de comunicacion
con el proveedor: Connection refused (connect failed)"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:56.241 [pool-1-thread-7] id=ev-09c
inventario.listarInventario -> host.docker.internal:6001 | intento 1/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:56.443 [pool-1-thread-7] id=ev-09c
inventario.listarInventario -> host.docker.internal:6001 | intento 2/3
```

```
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:56.645 [pool-1-thread-7] id=ev-09c
inventario.listarInventario -> host.docker.internal:6001 | intento 3/3
fallo: ERROR_INTERNO (Error de comunicacion con el proveedor: Connection
refused (connect failed))
[BROKER] 2026-11-14 15:18:56.645 [pool-1-thread-7] id=ev-09c
inventario.listarInventario -> host.docker.internal:6001 | ERROR
ERROR_INTERNO: Error de comunicacion con el proveedor: Connection refused
(connect failed) | 406 ms

Marca de no disponible en la bitacora:
[REGISTRO] 2026-11-14 15:18:56.645 [pool-1-thread-7] inventario marcado
como NO DISPONIBLE tras 3 fallos

## Paso 5: cuarta solicitud (rechazo inmediato, sin contactar al proveedor)

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-09d",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:57.3860881"
}
BROKER -> CLIENTE (13 ms)
{
  "idSolicitud": "ev-09d",
  "estado": "ERROR",
  "codigoError": "PROVEEDOR_NO_REGISTRADO",
  "mensajeError": "Proveedor no disponible: inventario",
  "fechaHora": "2026-11-14T15:18:57.418638718"
}
Resumen: estado=ERROR, codigo=PROVEEDOR_NO_REGISTRADO, mensaje="Proveedor
no disponible: inventario"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:57.418 [pool-1-thread-6] id=ev-09d
inventario.listarInventario -> host.docker.internal:6001 | ERROR
PROVEEDOR_NO_REGISTRADO: Proveedor no disponible: inventario | 0 ms

## Paso 6: vuelve el proveedor original (docker start uady-sistema-
centralizado) y se registra de nuevo
[REGISTRO] 2026-11-14 15:18:58.017 [pool-1-thread-1] registrar
gestorTiendas -> sistema-centralizado:6000
```

```
[REGISTRO] 2026-11-14 15:18:58.030 [pool-1-thread-15] registrar inventario
-> sistema-centralizado:6000
[REGISTRO] 2026-11-14 15:18:58.031 [pool-1-thread-2] registrar
gestorPrecios -> sistema-centralizado:6000

## Paso 7: el servicio vuelve a responder

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-09e",
  "nombreServicio": "inventario",
  "nombreOperacion": "listarInventario",
  "parametros": {},
  "fechaHora": "2026-11-14T09:18:59.5751667"
}
BROKER -> CLIENTE (95 ms)
{
  "idSolicitud": "ev-09e",
  "resultado": [
    {
      "numeroSerie": "SN-UADY-0001",
      "idProducto": 1,
      "nombreProducto": "Laptop Lenovo ThinkPad E14",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0006",
      "idProducto": 3,
      "nombreProducto": "Silla ergonomica",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0007",
      "idProducto": 4,
      "nombreProducto": "Impresora laser HP",
      "estado": "EN_ALMACEN"
    },
    {
      "numeroSerie": "SN-UADY-0009",
      "idProducto": 5,
      "nombreProducto": "Tablet Samsung Galaxy Tab",
      "estado": "EN_ALMACEN"
    }
  ],
  "estado": "EXITO",
```

```
  "fechaHora": "2026-11-14T15:18:59.688099393"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:59.688 [pool-1-thread-13] id=ev-09e
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 81 ms
```

Prueba 10: Pruebas automatizadas

Requisitos del Nivel 2 y preguntas de análisis

Evidencia: `10_pruebas_automatizadas.txt`

Resultado: `mvn test` con MariaDB: **195 pruebas, 0 fallos, 0 omitidas** (Common 10, Broker 47, Sistema 1 129, Sistema 2 9)

```
# Evidencia adicional: pruebas automatizadas
Fecha (hora local del equipo): 2026-11-14 09:21:33
Codigo: rama dev/chris, commit bb03b8c

Comando: mvn -B test, con la MariaDB de pruebas (DB_HOST=localhost,
DB_PORT=3307) para que se ejecuten tambien
las pruebas de integracion con la base de datos.

## Resultado por clase de prueba
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.067 s -- in mx.uady.inventario.common.contrato.SolicitudRespuestaTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.035 s -- in mx.uady.inventario.common.modelo.ModelosGsonTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.113 s -- in mx.uady.inventario.broker.network.BrokerServerTest
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.019 s -- in mx.uady.inventario.broker.registry.RegistroServiciosTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
1.845 s -- in mx.uady.inventario.broker.routing.EnrutadorTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.005 s -- in mx.uady.inventario.broker.util.BitacoraBrokerTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.005 s -- in mx.uady.inventario.broker.util.ConfiguracionBrokerTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.003 s -- in mx.uady.inventario.broker.util.JsonBrokerTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.867 s -- in mx.uady.inventario.central.controller.MercanciaControllerTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.052 s -- in mx.uady.inventario.central.controller.ProductoControllerTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.108 s -- in mx.uady.inventario.central.controller.TareaAsyncTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.467 s -- in mx.uady.inventario.central.controller.VentaControllerTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
```

```
0.008 s -- in mx.uady.inventario.central.network.DespachadorSolicitudesTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
3.096 s -- in mx.uady.inventario.central.network.IntegracionFase5Test
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.004 s -- in mx.uady.inventario.central.network.JsonServicioTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.078 s -- in mx.uady.inventario.central.network.RegistradorBrokerTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.015 s -- in mx.uady.inventario.central.network.ServidorServicioTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.002 s -- in mx.uady.inventario.central.network.VentaServicioTcpTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.003 s -- in mx.uady.inventario.central.repository.MercanciaRepositoryTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.081 s -- in
mx.uady.inventario.central.repository.VentaRepositoryIntegracionTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.076 s -- in
mx.uady.inventario.central.service.jdbc.ConsultasVentaJdbcIntegracionTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.185 s -- in
mx.uady.inventario.central.service.jdbc.MercanciaServiceJdbcIntegracionTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.102 s -- in
mx.uady.inventario.central.service.jdbc.ProductoServiceJdbcIntegracionTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.009 s -- in mx.uady.inventario.central.service.jdbc.TransaccionesTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.081 s -- in
mx.uady.inventario.central.service.memoria.MercanciaServiceMemoriaTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.005 s -- in
mx.uady.inventario.central.service.memoria.ProductoServiceMemoriaTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.002 s -- in mx.uady.inventario.central.service.memoria.VentasMemoriaTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.002 s -- in mx.uady.inventario.central.service.ProductoValidatorTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.021 s -- in mx.uady.inventario.central.service.QrServiceTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.004 s -- in mx.uady.inventario.central.service.VentaServiceTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.011 s -- in mx.uady.inventario.central.util.ConexionBDIntegracionTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.002 s -- in mx.uady.inventario.central.util.ConfiguracionBDTest
```

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.004 s -- in mx.uady.inventario.central.util.ConfiguracionServicioTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.002 s -- in mx.uady.inventario.central.view.FiltroTextoTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.002 s -- in mx.uady.inventario.central.view.MensajesTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.024 s -- in mx.uady.inventario.central.view.MercanciaPanelTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.001 s -- in mx.uady.inventario.central.view.MercanciaTableModelTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.003 s -- in mx.uady.inventario.central.view.ProductoDialogTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.076 s -- in mx.uady.inventario.central.view.ProductosPanelTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.015 s -- in mx.uady.inventario.central.view.VentaHistorialPanelTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.014 s -- in mx.uady.inventario.central.view.VentaPanelTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.286 s -- in
mx.uady.inventario.cliente.controller.RegistroVentaControllerTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.008 s -- in mx.uady.inventario.cliente.util.ConfiguracionNetworkTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed:
0.054 s -- in mx.uady.inventario.cliente.view.RegistroVentaFrameTest

## Resultado por modulo
[INFO] Reactor Build Order:
[INFO] Building Sistema de Trazabilidad e Inventarios UADY 1.0.0-SNAPSHOT
[1/5]
[INFO] Building Common - Modulo comun 1.0.0-SNAPSHOT
[2/5]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building Broker - Intermediario 1.0.0-SNAPSHOT
[3/5]
[INFO] Tests run: 47, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building Sistema 1 - Centralizado / Proveedor 1.0.0-SNAPSHOT
[4/5]
[INFO] Tests run: 129, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building Sistema 2 - Cliente del Proveedor 1.0.0-SNAPSHOT
[5/5]
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Prueba 11: Atención concurrente con hilos

Requisitos del Nivel 2 y preguntas de análisis

Evidencia: `11_concurrencia_hilos.txt`

Resultado: 20 clientes simultáneos: **20/20** reciben `EXITO` con el id de su propia solicitud; los atendieron **16 hilos distintos** del pool

```
# Evidencia adicional: atencion concurrente con hilos
Fecha (hora local del equipo): 2026-11-14 09:18:19
Codigo: rama dev/chris, commit bb03b8c

20 clientes abren cada uno su propio socket y envian una solicitud al mismo
tiempo. El Broker atiende
cada conexion en un hilo de su pool (16 hilos) y cada cliente recibe la
respuesta con el id de SU
solicitud. La bitacora muestra los distintos hilos del pool que atendieron.

## Clientes
ev-11-01  inventario.listarInventario    -> idRespuesta=ev-11-01
(coincide)  EXITO  70 ms
ev-11-02  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-02
(coincide)  EXITO  70 ms
ev-11-03  inventario.listarInventario    -> idRespuesta=ev-11-03
(coincide)  EXITO  70 ms
ev-11-04  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-04
(coincide)  EXITO  70 ms
ev-11-05  inventario.listarInventario    -> idRespuesta=ev-11-05
(coincide)  EXITO  82 ms
ev-11-06  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-06
(coincide)  EXITO  70 ms
ev-11-07  inventario.listarInventario    -> idRespuesta=ev-11-07
(coincide)  EXITO  70 ms
ev-11-08  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-08
(coincide)  EXITO  79 ms
ev-11-09  inventario.listarInventario    -> idRespuesta=ev-11-09
(coincide)  EXITO  70 ms
ev-11-10  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-10
(coincide)  EXITO  70 ms
ev-11-11  inventario.listarInventario    -> idRespuesta=ev-11-11
(coincide)  EXITO  70 ms
ev-11-12  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-12
```

```
(coincide)  EXITO  79 ms
ev-11-13  inventario.listarInventario    -> idRespuesta=ev-11-13
(coincide)  EXITO  70 ms
ev-11-14  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-14
(coincide)  EXITO  70 ms
ev-11-15  inventario.listarInventario    -> idRespuesta=ev-11-15
(coincide)  EXITO  82 ms
ev-11-16  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-16
(coincide)  EXITO  70 ms
ev-11-17  inventario.listarInventario    -> idRespuesta=ev-11-17
(coincide)  EXITO  70 ms
ev-11-18  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-18
(coincide)  EXITO  70 ms
ev-11-19  inventario.listarInventario    -> idRespuesta=ev-11-19
(coincide)  EXITO  73 ms
ev-11-20  gestorTiendas.listarTiendas    -> idRespuesta=ev-11-20
(coincide)  EXITO  70 ms

Resumen: 20/20 clientes recibieron EXITO con el id de su propia solicitud;
tiempo total 83 ms

## Bitacora del Broker
[BROKER] 2026-11-14 15:18:20.337 [pool-1-thread-1] id=ev-11-20
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 11 ms
[BROKER] 2026-11-14 15:18:20.338 [pool-1-thread-11] id=ev-11-02
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 12 ms
[BROKER] 2026-11-14 15:18:20.338 [pool-1-thread-9] id=ev-11-16
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 13 ms
[BROKER] 2026-11-14 15:18:20.340 [pool-1-thread-3] id=ev-11-10
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 14 ms
[BROKER] 2026-11-14 15:18:20.340 [pool-1-thread-16] id=ev-11-07
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 15 ms
[BROKER] 2026-11-14 15:18:20.340 [pool-1-thread-4] id=ev-11-06
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 13 ms
[BROKER] 2026-11-14 15:18:20.341 [pool-1-thread-7] id=ev-11-09
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 16 ms
[BROKER] 2026-11-14 15:18:20.343 [pool-1-thread-14] id=ev-11-04
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 17 ms
[BROKER] 2026-11-14 15:18:20.343 [pool-1-thread-13] id=ev-11-14
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 15 ms
[BROKER] 2026-11-14 15:18:20.343 [pool-1-thread-5] id=ev-11-13
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 17 ms
[BROKER] 2026-11-14 15:18:20.344 [pool-1-thread-8] id=ev-11-18
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 18 ms
[BROKER] 2026-11-14 15:18:20.343 [pool-1-thread-6] id=ev-11-01
```

```
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 16 ms
[BROKER] 2026-11-14 15:18:20.345 [pool-1-thread-2] id=ev-11-03
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 19 ms
[BROKER] 2026-11-14 15:18:20.347 [pool-1-thread-15] id=ev-11-11
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 19 ms
[BROKER] 2026-11-14 15:18:20.347 [pool-1-thread-10] id=ev-11-17
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 20 ms
[BROKER] 2026-11-14 15:18:20.351 [pool-1-thread-12] id=ev-11-19
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 24 ms
[BROKER] 2026-11-14 15:18:20.359 [pool-1-thread-15] id=ev-11-08
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 6 ms
[BROKER] 2026-11-14 15:18:20.359 [pool-1-thread-6] id=ev-11-12
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 6 ms
[BROKER] 2026-11-14 15:18:20.361 [pool-1-thread-13] id=ev-11-15
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 9 ms
[BROKER] 2026-11-14 15:18:20.362 [pool-1-thread-2] id=ev-11-05
inventario.listarInventario -> sistema-centralizado:6000 | EXITO | 9 ms

Hilos distintos del pool que atendieron: 16
Prueba automatizada relacionada: IntegracionFase5Test
(veinteClientesSimultaneosATravesDelBrokerTerminanSinErrores).
```

Prueba 12: Pregunta 3 y 4: el Broker deja de funcionar / punto único de falla

Requisitos del Nivel 2 y preguntas de análisis

Evidencia: `12_broker_caido.txt`

Resultado: Sin Broker, el cliente no llega a ningún servicio aunque el Sistema 1 siga vivo. Al reiniciar el Broker, su registro en memoria está vacío (`SERVICIO_DESCONOCIDO`) hasta que el Sistema 1 se vuelve a registrar

```
# Evidencia adicional: el Broker deja de funcionar (preguntas 3 y 4)
Fecha (hora local del equipo): 2026-11-14 09:18:59
Codigo: rama dev/chris, commit bb03b8c

Se detiene el Broker. El cliente no puede comunicarse con ningun servicio
aunque el Sistema 1 siga
funcionando: el Broker es un punto unico de falla. Al reiniciarlo, su
registro (en memoria) esta vacio y
el Sistema 1 no se vuelve a registrar por si solo, asi que los servicios
aparecen como desconocidos hasta
reiniciar el proveedor. Esta limitacion es una mejora pendiente (re-
registro periodico, error #2).

## Paso 1: docker stop uady-broker
## Paso 2: el cliente intenta una solicitud

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-12a",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "listarTiendas",
  "parametros": {},
  "fechaHora": "2026-11-14T09:19:00.6149458"
}
SIN RESPUESTA (13 ms): ConnectException: Connection refused: getsockopt
Resumen: el cliente no pudo comunicarse con el Broker

Bitacora del Broker (docker logs uady-broker):

## Paso 3: el Sistema 1 sigue en ejecucion (docker ps)
uady-sistema-centralizado | Up 3 seconds
```

```
uady-mariadb | Up 22 seconds (healthy)
uady-mariadb-pruebas | Up 24 minutes (healthy)

## Paso 4: docker start uady-broker
## Paso 5: el Broker volvio, pero su registro esta vacio

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-12b",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "listarTiendas",
  "parametros": {},
  "fechaHora": "2026-11-14T09:19:06.6972985"
}
BROKER -> CLIENTE (75 ms)
{
  "idSolicitud": "ev-12b",
  "estado": "ERROR",
  "codigoError": "SERVICIO_DESCONOCIDO",
  "mensajeError": "Servicio desconocido: gestorTiendas",
  "fechaHora": "2026-11-14T15:19:06.779518031"
}
Resumen: estado=ERROR, codigo=SERVICIO_DESCONOCIDO, mensaje="Servicio
desconocido: gestorTiendas"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:19:06.787 [pool-1-thread-2] id=ev-12b
gestorTiendas.listarTiendas -> (sin proveedor) | ERROR
SERVICIO_DESCONOCIDO: Servicio desconocido: gestorTiendas | 46 ms

## Paso 6: se reinicia el Sistema 1, que se registra al arrancar

## Paso 7: el servicio vuelve a responder

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-12c",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "listarTiendas",
  "parametros": {},
  "fechaHora": "2026-11-14T09:19:09.147447"
}
BROKER -> CLIENTE (107 ms)
{
  "idSolicitud": "ev-12c",
```

```
  "resultado": [
    {
      "idTienda": 1,
      "nombre": "Almacen Central UADY",
      "direccion": "Calle 60 s/n, Centro, Merida, Yucatan"
    },
    {
      "idTienda": 2,
      "nombre": "Tienda Facultad de Ingenieria",
      "direccion": "Periferico Norte, Campus de Ingenieria, Merida"
    },
    {
      "idTienda": 3,
      "nombre": "Tienda Facultad de Matematicas",
      "direccion": "Campus de Ciencias Exactas, Merida"
    }
  ],
  "estado": "EXITO",
  "fechaHora": "2026-11-14T15:19:09.27751569"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:19:09.278 [pool-1-thread-6] id=ev-12c
gestorTiendas.listarTiendas -> sistema-centralizado:6000 | EXITO | 88 ms
```

Prueba 13: Pregunta 9: solicitudes duplicadas

Requisitos del Nivel 2 y preguntas de análisis

Evidencia: `13_solicitud_duplicada.txt`

Resultado: La misma venta dos veces: la segunda se rechaza (`ya fue vendida`) y en la BD queda **una sola** venta

```
# Evidencia adicional: solicitud duplicada (pregunta 9)
Fecha (hora local del equipo): 2026-11-14 09:18:18
Codigo: rama dev/chris, commit bb03b8c

La misma venta se envia dos veces. La primera se registra; la segunda se
rechaza porque la unidad ya
esta VENDIDO. En la base de datos queda una sola venta: la transaccion con
bloqueo pesimista
(SELECT ... FOR UPDATE) evita que una solicitud repetida registre la venta
dos veces.

## Primer envio

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-13a",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "actualizarTiendaVenta",
  "parametros": {
    "numero_serie": "SN-UADY-0005",
    "id_tienda": 2,
    "id_usuario": 4
  },
  "fechaHora": "2026-11-14T09:18:18.9463501"
}
BROKER -> CLIENTE (26 ms)
{
  "idSolicitud": "ev-13a",
  "resultado": {
    "numero_serie": "SN-UADY-0005",
    "id_tienda": 2,
    "fecha_venta": "2026-11-14T15:18:18"
  },
  "estado": "EXITO",
```

```
  "fechaHora": "2026-11-14T15:18:18.996737669"
}
Resumen: estado=EXITO

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:18.997 [pool-1-thread-16] id=ev-13a
gestorTiendas.actualizarTiendaVenta -> sistema-centralizado:6000 | EXITO |
10 ms

## Segundo envio (misma venta, id de solicitud distinto)

CLIENTE -> BROKER (localhost:5000)
{
  "idSolicitud": "ev-13b",
  "nombreServicio": "gestorTiendas",
  "nombreOperacion": "actualizarTiendaVenta",
  "parametros": {
    "numero_serie": "SN-UADY-0005",
    "id_tienda": 2,
    "id_usuario": 4
  },
  "fechaHora": "2026-11-14T09:18:19.5111541"
}
BROKER -> CLIENTE (20 ms)
{
  "idSolicitud": "ev-13b",
  "estado": "ERROR",
  "codigoError": "PARAMETROS_INVALIDOS",
  "mensajeError": "La mercancía ya fue vendida",
  "fechaHora": "2026-11-14T15:18:19.549968198"
}
Resumen: estado=ERROR, codigo=PARAMETROS_INVALIDOS, mensaje="La mercancía
ya fue vendida"

Bitacora del Broker (docker logs uady-broker):
[BROKER] 2026-11-14 15:18:19.550 [pool-1-thread-1] id=ev-13b
gestorTiendas.actualizarTiendaVenta -> sistema-centralizado:6000 | ERROR
PARAMETROS_INVALIDOS: La mercancía ya fue vendida | 6 ms

Ventas registradas para SN-UADY-0005 en la base de datos:
ventas
1
```

Prueba 14: Pregunta 7: costo en rendimiento

Requisitos del Nivel 2 y preguntas de análisis

Evidencia: `14_costo_rendimiento.txt`

Resultado: 100 solicitudes por camino: directo 3.53 ms, por el Broker 4.45 ms de promedio → **+0.92 ms por solicitud**

```
# Evidencia adicional: costo del Broker en rendimiento (pregunta 7)
Fecha (hora local del equipo): 2026-11-14 09:18:20
Codigo: rama dev/chris, commit bb03b8c

La misma operacion se envia 100 veces directo al Sistema 1 (puerto 6000,
publicado por Docker solo
para esta medicion) y 100 veces a traves del Broker (puerto 5000). La
diferencia es el costo del salto
adicional: una conexion TCP mas, marshalling/unmarshalling extra y la
busqueda en el registro.

Operacion: gestorTiendas.listarTiendas, 100 solicitudes secuenciales por
camino, una conexion TCP nueva por solicitud.

Camino                              promedio    mediana     minimo
maximo
Directo al Sistema 1 (:6000)          3.53 ms     3.19 ms     2.73 ms
11.46 ms
A traves del Broker (:5000)           4.45 ms     4.07 ms     3.35 ms
11.27 ms

Resumen: el Broker agrega en promedio 0.92 ms por solicitud (26.2% mas que
la llamada directa).
```

Observaciones

- **Hora UTC en los contenedores.** Los contenedores corren en UTC, así que la bitácora y la `fecha_venta` muestran 6 horas más que la hora local del equipo (por ejemplo, 15:14 en lugar de 09:14). La fecha en el encabezado de cada archivo es la hora local. Es una limitación conocida (zona horaria, error #6).
- **Timeout del cliente frente al Broker** (`07` , caso B). Cuando el proveedor no responde, el Broker tarda unos 15 s (3 intentos × 5 s) en contestar `TIMEOUT` ; el Sistema 2 espera 5 s, así que el usuario vería antes su propio timeout. Es una limitación conocida (error #1).
- **Re-registro tras reiniciar el Broker** (`12`). El Sistema 1 solo se registra al arrancar. Es una limitación conocida (error #2); la mejora es un re-registro periódico.

Cómo reproducirlas

Requisitos: Docker Desktop, JDK 21+, Maven y Git Bash. Los pasos 1 y 2 se ejecutan en la raíz de un checkout con el código (rama `qa` o `dev/chris`, que incluyen la bitácora del Broker):

1. `mvn -B package -DskipTests` (los clientes usan `sistema-cliente/target/sistema-cliente.jar` como classpath).
2. `docker compose -p evidencias up --build -d mariadb broker sistema-centralizado`
3. Desde la carpeta `docs/02_documentacion_final/markdown/04_evidencias_pruebas`, en Git Bash: `bash parte1.sh`, `bash parte2.sh` y `bash parte3.sh`. Los scripts buscan el código cuatro carpetas arriba de esta; si esta carpeta está en otro checkout (por ejemplo, en la rama `docs`), indica la ruta del código: `REPO=/ruta/al/checkout bash parte1.sh`.
4. `parte1.sh`: 01–06, 11, 13 y 14 (stack sano).
5. `parte2.sh`: 08, 09 y 12 (detiene y reinicia componentes; levanta un segundo proveedor en el puerto 6001).
6. `parte3.sh`: 07 (excepción del proveedor y base de datos detenida).
7. La 10 se obtiene con `mvn -B test` y las variables `DB_*` apuntando a una MariaDB con el esquema.
8. Al terminar: `docker compose -p evidencias down -v`.

Los scripts regeneran los archivos `.txt`; para que el documento muestre los nuevos logs hay que copiarlos en la sección de la prueba correspondiente.

Archivos de apoyo

Archivo	Uso
`EvidenciaCliente.java`	Envía una solicitud al Broker y muestra el JSON enviado y recibido
`EvidenciaConcurrencia.java`	N clientes simultáneos; verifica que cada uno reciba su propia respuesta
`EvidenciaRendimiento.java`	Mide la misma operación directo al proveedor y a través del Broker
`comun.sh`	Funciones compartidas por los scripts de evidencias
`parte1.sh`	Genera las evidencias 01–06, 11, 13 y 14 (stack sano)
`parte2.sh`	Genera las evidencias 08, 09 y 12; deja la consola del segundo proveedor en `proveedor2.log`
`parte3.sh`	Genera la evidencia 07 (excepción del proveedor y base de datos detenida)

EvidenciaCliente.java

Envía una solicitud al Broker y muestra el JSON enviado y recibido.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
```

```
import com.google.gson.GsonBuilder;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

import mx.uady.inventario.common.contrato.Respuesta;
import mx.uady.inventario.common.contrato.Solicitud;
import mx.uady.inventario.common.json.GsonFactory;

/**
 * Cliente de evidencias: envía UNA solicitud al Broker por socket TCP y
muestra
 * el JSON enviado y el recibido. Solo conoce el Broker (host y puerto),
nunca al proveedor.
 *
 * Uso: java EvidenciaCliente <idSolicitud> <servicio> <operacion>
<parametrosJson>
 * El destino es localhost:5000; se puede cambiar con -Dhost=... y
-Dpuerto=...
 * El tiempo de espera de la respuesta es 15000 ms; se puede cambiar con
-DtimeoutMs=...
 */
public class EvidenciaCliente {

    public static void main(String[] args) throws Exception {
        String host = System.getProperty("host", "localhost");
        int puerto = Integer.getInteger("puerto", 5000);
        String id = args[0];
        JsonObject parametros =
JsonParser.parseString(args[3]).getAsJsonObject();
        Solicitud solicitud = Solicitud.crear(id, args[1], args[2],
parametros);

        String json =
GsonFactory.DEFAULT.toJson(solicitud);               // marshalling
        System.out.println("CLIENTE -> BROKER (" + host + ":" + puerto +
")");
        System.out.println(bonito(json));

        long inicio = System.nanoTime();
        try (Socket socket = new Socket()) {
            socket.connect(new InetSocketAddress(host, puerto), 3000);
            socket.setSoTimeout(Integer.getInteger("timeoutMs", 15_000));
            PrintWriter salida = new PrintWriter(socket.getOutputStream(),
true, StandardCharsets.UTF_8);
            BufferedReader entrada = new BufferedReader(
```

```
                    new InputStreamReader(socket.getInputStream(),
StandardCharsets.UTF_8));
            salida.println(json);
            String recibido = entrada.readLine();
            long ms = (System.nanoTime() - inicio) / 1_000_000;

            Respuesta respuesta = GsonFactory.DEFAULT.fromJson(recibido,
Respuesta.class); // unmarshalling
            System.out.println("BROKER -> CLIENTE (" + ms + " ms)");
            System.out.println(bonito(recibido));
            System.out.println("Resumen: estado=" + respuesta.getEstado()
                    + (respuesta.getCodigoError() == null ? ""
                            : ", codigo=" + respuesta.getCodigoError() + ",
mensaje=\"" + respuesta.getMensajeError() + "\""));
        } catch (IOException e) {
            long ms = (System.nanoTime() - inicio) / 1_000_000;
            System.out.println("SIN RESPUESTA (" + ms + " ms): " +
e.getClass().getSimpleName() + ": " + e.getMessage());
            System.out.println("Resumen: el cliente no pudo comunicarse con
el Broker");
        }
    }

    private static String bonito(String json) {
        return new
GsonBuilder().setPrettyPrinting().disableHtmlEscaping().create()
                .toJson(JsonParser.parseString(json));
    }
}
```

EvidenciaConcurrencia.java

N clientes simultáneos; verifica que cada uno reciba su propia respuesta.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
```

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

import mx.uady.inventario.common.contrato.Respuesta;
import mx.uady.inventario.common.contrato.Solicitud;
import mx.uady.inventario.common.json.GsonFactory;

/**
 * Evidencia de atención concurrente: N clientes envían al mismo tiempo una
solicitud al Broker
 * (cada uno con su propio socket e id) y se comprueba que cada uno recibe
SU respuesta.
 *
 * Uso: java EvidenciaConcurrencia <cantidad> <prefijoId>
 */
public class EvidenciaConcurrencia {

    public static void main(String[] args) throws Exception {
        int cantidad = Integer.parseInt(args[0]);
        String prefijo = args[1];
        String[] operaciones = {"gestorTiendas/listarTiendas", "inventario/
listarInventario"};

        ExecutorService ejecutor = Executors.newFixedThreadPool(cantidad);
        CountDownLatch salida = new CountDownLatch(1);
        List<Future<String>> futuros = new ArrayList<>();
        for (int i = 1; i <= cantidad; i++) {
            String id = prefijo + "-" + String.format("%02d", i);
            String[] destino = operaciones[i %
operaciones.length].split("/");
            futuros.add(ejecutor.submit(() -> {
                salida.await();
                long inicio = System.nanoTime();
                Solicitud solicitud = Solicitud.crear(id, destino[0],
destino[1], null);
                try (Socket socket = new Socket()) {
                    socket.connect(new InetSocketAddress("localhost",
5000), 3000);
                    socket.setSoTimeout(15_000);
                    PrintWriter out = new
PrintWriter(socket.getOutputStream(), true, StandardCharsets.UTF_8);
                    BufferedReader in = new BufferedReader(
                            new InputStreamReader(socket.getInputStream(),
StandardCharsets.UTF_8));
```

```
                    out.println(GsonFactory.DEFAULT.toJson(solicitud));
                    Respuesta r =
GsonFactory.DEFAULT.fromJson(in.readLine(), Respuesta.class);
                    long ms = (System.nanoTime() - inicio) / 1_000_000;
                    String coincide = id.equals(r.getIdSolicitud()) ?
"coincide" : "NO COINCIDE";
                    return String.format("%s  %-30s -> idRespuesta=%s (%s)
%s  %d ms",
                            id, destino[0] + "." + destino[1],
r.getIdSolicitud(), coincide, r.getEstado(), ms);
                }
            }));
        }

        long inicio = System.nanoTime();
        salida.countDown();   // todos los clientes arrancan a la vez
        int correctas = 0;
        for (Future<String> f : futuros) {
            String linea = f.get();
            System.out.println(linea);
            if (linea.contains("(coincide)") && linea.contains("EXITO")) {
                correctas++;
            }
        }
        long total = (System.nanoTime() - inicio) / 1_000_000;
        ejecutor.shutdown();
        System.out.println();
        System.out.println("Resumen: " + correctas + "/" + cantidad
                + " clientes recibieron EXITO con el id de su propia
solicitud; tiempo total " + total + " ms");
    }
}
```

EvidenciaRendimiento.java

Mide la misma operación directo al proveedor y a través del Broker.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
```

```
import java.util.Arrays;

import mx.uady.inventario.common.contrato.EstadoRespuesta;
import mx.uady.inventario.common.contrato.Respuesta;
import mx.uady.inventario.common.contrato.Solicitud;
import mx.uady.inventario.common.json.GsonFactory;

/**
 * Evidencia del costo del Broker en rendimiento: mide la misma operación
enviada N veces
 * directamente al proveedor (Sistema 1) y N veces a través del Broker, con
una conexión
 * nueva por solicitud en ambos casos (igual que el cliente real).
 *
 * Uso: java EvidenciaRendimiento <repeticiones> <puertoProveedor>
<puertoBroker>
 */
public class EvidenciaRendimiento {

    public static void main(String[] args) throws Exception {
        int n = Integer.parseInt(args[0]);
        int puertoProveedor = Integer.parseInt(args[1]);
        int puertoBroker = Integer.parseInt(args[2]);

        // Calentamiento: las primeras solicitudes incluyen la carga de
clases de la JVM.
        medir(5, puertoProveedor, "calentamiento-directo");
        medir(5, puertoBroker, "calentamiento-broker");

        double[] directo = medir(n, puertoProveedor, "rend-directo");
        double[] broker = medir(n, puertoBroker, "rend-broker");

        System.out.println("Operacion: gestorTiendas.listarTiendas, " + n +
" solicitudes secuenciales por camino,"
                + " una conexion TCP nueva por solicitud.");
        System.out.println();
        System.out.printf("%-34s %10s %10s %10s %10s%n", "Camino",
"promedio", "mediana", "minimo", "maximo");
        imprimir("Directo al Sistema 1 (:" + puertoProveedor + ")",
directo);
        imprimir("A traves del Broker (:" + puertoBroker + ")", broker);
        System.out.println();
        System.out.printf("Resumen: el Broker agrega en promedio %.2f ms
por solicitud (%.1f%% mas que la llamada directa).%n",
                promedio(broker) - promedio(directo), (promedio(broker) /
```

```
promedio(directo) - 1) * 100);
    }

    private static double[] medir(int n, int puerto, String prefijo) throws
Exception {
        double[] tiempos = new double[n];
        for (int i = 0; i < n; i++) {
            Solicitud s = Solicitud.crear(prefijo + "-" + i,
"gestorTiendas", "listarTiendas", null);
            long inicio = System.nanoTime();
            try (Socket socket = new Socket()) {
                socket.connect(new InetSocketAddress("localhost", puerto),
3000);
                socket.setSoTimeout(15_000);
                PrintWriter out = new PrintWriter(socket.getOutputStream(),
true, StandardCharsets.UTF_8);
                BufferedReader in = new BufferedReader(
                        new InputStreamReader(socket.getInputStream(),
StandardCharsets.UTF_8));
                out.println(GsonFactory.DEFAULT.toJson(s));
                Respuesta r = GsonFactory.DEFAULT.fromJson(in.readLine(),
Respuesta.class);
                if (r.getEstado() != EstadoRespuesta.EXITO) {
                    throw new IllegalStateException("Respuesta de error en
" + puerto + ": " + r.getMensajeError());
                }
            }
            tiempos[i] = (System.nanoTime() - inicio) / 1_000_000.0;
        }
        return tiempos;
    }

    private static void imprimir(String camino, double[] t) {
        double[] orden = t.clone();
        Arrays.sort(orden);
        System.out.printf("%-34s %8.2f ms %8.2f ms %8.2f ms %8.2f ms%n",
camino, promedio(t),
                orden[orden.length / 2], orden[0], orden[orden.length -
1]);
    }

    private static double promedio(double[] t) {
        return Arrays.stream(t).average().orElse(0);
    }
}
```

comun.sh

Funciones compartidas por los scripts de evidencias.

```
#!/usr/bin/env bash
# Funciones compartidas por los scripts de evidencias.
# Requiere: el stack de docker compose arriba (uady-mariadb, uady-broker,
uady-sistema-centralizado)
# y el proyecto compilado (mvn package) para usar el jar del cliente como
classpath.

EV="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
# Raíz del checkout con el código (rama qa o dev/chris). Por defecto,
cuatro niveles arriba de esta
# carpeta (docs/02_documentacion_final/markdown/04_evidencias_pruebas); se
puede indicar otra con REPO=/ruta/al/repo.
REPO="${REPO:-$(cd "$EV/../../../.." && pwd)}"
CP="$REPO/sistema-cliente/target/sistema-cliente.jar"
JAVA="java -Dstdout.encoding=UTF-8 -Dfile.encoding=UTF-8"

# Envía una solicitud al Broker: cli <id> <servicio> <operacion>
<parametrosJson>
cli() { $JAVA -cp "$CP" "$EV/EvidenciaCliente.java" "$@" 2>&1 | grep -v
'^WARNING'; }

# Líneas de la bitácora del Broker de una solicitud: bitacora <id>
bitacora() { docker logs uady-broker 2>&1 | grep -F "id=$1 "; }

# Encabezado de un archivo de evidencia: encabezado <archivo> <titulo>
<descripcion>
encabezado() {
  {
    echo "# $2"
    echo "Fecha (hora local del equipo): $(date '+%Y-%m-%d %H:%M:%S')"
    echo "Codigo: rama dev/chris, commit $(git -C "$REPO" rev-parse --short
HEAD)"
    echo
    echo "$3"
    echo
  } > "$EV/$1"
}

# Agrega una solicitud completa (JSON enviado y recibido) y su bitácora a
un archivo:
```

```
# paso <archivo> <titulo del paso> <id> <servicio> <operacion>
<parametrosJson>
paso() {
  local archivo="$1" titulo="$2"; shift 2
  {
    echo "## $titulo"
    echo
    cli "$@"
    echo
    echo "Bitacora del Broker (docker logs uady-broker):"
    bitacora "$1"
    echo
  } >> "$EV/$archivo"
}

# Consulta SQL en la base del stack: sql "<consulta>"
sql() { docker exec uady-mariadb sh -c "mariadb -uroot
-p\"$MARIADB_ROOT_PASSWORD\" -D \"$MARIADB_DATABASE\" -e \"$1\""; }
```

parte1.sh

Genera las evidencias 01–06, 11, 13 y 14 (stack sano).

```
#!/usr/bin/env bash
# Parte 1: evidencias con el stack sano (01-06, 11, 13 y 14).
source "$(dirname "$0")/comun.sh"

# 01 Registro de proveedores
encabezado 01_registro_proveedores.txt "Prueba 1: Registro correcto de
proveedores" \
"Al arrancar, el Sistema 1 (proveedor) se registra dinamicamente en el
Broker enviando la operacion
broker.registrarProveedor por socket, una vez por cada servicio que ofrece.
El Broker asocia cada
nombre de servicio con el host y puerto del proveedor."
{
  echo "## Consola del Sistema 1 (docker logs uady-sistema-centralizado)"
  docker logs uady-sistema-centralizado 2>&1 | grep -E "Servicios:|
Proveedor registrado|Escuchando"
  echo
  echo "## Bitacora del Broker (docker logs uady-broker)"
  docker logs uady-broker 2>&1 | grep -E "^Broker iniciado|
registrarProveedor|\[REGISTRO\] .* registrar "
```

```
  echo
  echo "Resumen: los 3 servicios (gestorTiendas, inventario, gestorPrecios)
quedan registrados hacia"
  echo "sistema-centralizado:6000. Prueba automatizada relacionada:
RegistroServiciosTest (registrarYBuscar)."
} >> "$EV/01_registro_proveedores.txt"

# 02 Localizacion
encabezado 02_localizacion_servicio.txt "Prueba 2: Localizacion de un
servicio" \
"El cliente solo conoce la direccion del Broker (localhost:5000). El Broker
busca en su registro el
proveedor del servicio 'inventario' y le reenvia la solicitud. La bitacora
muestra el proveedor localizado."
paso 02_localizacion_servicio.txt "inventario.listarInventario" ev-02
inventario listarInventario '{}'

# 03 Una operacion por servicio
encabezado 03_ejecucion_operacion.txt "Prueba 3: Ejecucion de una operacion
(una por cada servicio registrado)" \
"Se ejecuta una operacion de cada uno de los 3 servicios registrados. Todas
viajan por el Broker
como JSON por socket y llegan al proveedor localizado."
paso 03_ejecucion_operacion.txt "Servicio gestorTiendas: listarTiendas"
ev-03a gestorTiendas listarTiendas '{}'
paso 03_ejecucion_operacion.txt "Servicio inventario: listarInventario"
ev-03b inventario listarInventario '{}'
paso 03_ejecucion_operacion.txt "Servicio gestorPrecios:
listarPreciosProducto" ev-03c gestorPrecios listarPreciosProducto
'{"id_producto":1}'

# 04 Respuesta exitosa
encabezado 04_respuesta_exitosa.txt "Prueba 4: Respuesta exitosa" \
"El cliente registra la venta de una unidad (CU2). El Sistema 1 la procesa
en una transaccion y la
respuesta incluye id de la solicitud, resultado, estado y fecha y hora."
paso 04_respuesta_exitosa.txt "gestorTiendas.actualizarTiendaVenta" ev-04
gestorTiendas actualizarTiendaVenta \
  '{"numero_serie":"SN-UADY-0003","id_tienda":1,"id_usuario":2}'
{
  echo "Estado en la base de datos:"
  sql "SELECT m.numero_serie, m.estado, v.id_tienda, v.id_usuario,
v.fecha_venta FROM mercancia m JOIN ventas v ON v.numero_serie =
m.numero_serie WHERE m.numero_serie = 'SN-UADY-0003'"
} >> "$EV/04_respuesta_exitosa.txt"
```

```
# 05 Servicio desconocido
encabezado 05_servicio_desconocido.txt "Prueba 5: Servicio desconocido" \
"El cliente pide un servicio que ningun proveedor registro. El Broker lo
rechaza sin contactar a
ningun proveedor y sin caerse."
paso 05_servicio_desconocido.txt "servicioFantasma.listar" ev-05
servicioFantasma listar '{}'

# 06 Parametro invalido
encabezado 06_parametro_invalido.txt "Prueba 6: Parametro invalido" \
"Se envian ventas con parametros faltantes o invalidos. El proveedor los
valida y responde
PARAMETROS_INVALIDOS con un mensaje claro, conservando el id de la
solicitud."
paso 06_parametro_invalido.txt "Falta numero_serie" ev-06a gestorTiendas
actualizarTiendaVenta '{"id_tienda":1,"id_usuario":2}'
paso 06_parametro_invalido.txt "id_tienda negativo" ev-06b gestorTiendas
actualizarTiendaVenta '{"numero_serie":"SN-
UADY-0001","id_tienda":-5,"id_usuario":2}'
paso 06_parametro_invalido.txt "Producto inexistente en gestorPrecios"
ev-06c gestorPrecios listarPreciosProducto '{"id_producto":999}'

# 13 Solicitud duplicada (pregunta 9)
encabezado 13_solicitud_duplicada.txt "Evidencia adicional: solicitud
duplicada (pregunta 9)" \
"La misma venta se envia dos veces. La primera se registra; la segunda se
rechaza porque la unidad ya
esta VENDIDO. En la base de datos queda una sola venta: la transaccion con
bloqueo pesimista
(SELECT ... FOR UPDATE) evita que una solicitud repetida registre la venta
dos veces."
paso 13_solicitud_duplicada.txt "Primer envio" ev-13a gestorTiendas
actualizarTiendaVenta '{"numero_serie":"SN-
UADY-0005","id_tienda":2,"id_usuario":4}'
paso 13_solicitud_duplicada.txt "Segundo envio (misma venta, id de
solicitud distinto)" ev-13b gestorTiendas actualizarTiendaVenta
'{"numero_serie":"SN-UADY-0005","id_tienda":2,"id_usuario":4}'
{
  echo "Ventas registradas para SN-UADY-0005 en la base de datos:"
  sql "SELECT COUNT(*) AS ventas FROM ventas WHERE numero_serie = 'SN-
UADY-0005'"
} >> "$EV/13_solicitud_duplicada.txt"

# 11 Concurrencia
```

```
encabezado 11_concurrencia_hilos.txt "Evidencia adicional: atencion
concurrente con hilos" \
"20 clientes abren cada uno su propio socket y envian una solicitud al
mismo tiempo. El Broker atiende
cada conexion en un hilo de su pool (16 hilos) y cada cliente recibe la
respuesta con el id de SU
solicitud. La bitacora muestra los distintos hilos del pool que
atendieron."
{
  echo "## Clientes"
  $JAVA -cp "$CP" "$EV/EvidenciaConcurrencia.java" 20 ev-11 2>&1 | grep -v
'^WARNING'
  echo
  echo "## Bitacora del Broker"
  docker logs uady-broker 2>&1 | grep -F "id=ev-11-"
  echo
  echo "Hilos distintos del pool que atendieron: $(docker logs uady-broker
2>&1 | grep -F 'id=ev-11-' | grep -o 'pool-1-thread-[0-9]*' | sort -u | wc
-l)"
  echo "Prueba automatizada relacionada: IntegracionFase5Test
(veinteClientesSimultaneosATravesDelBrokerTerminanSinErrores)."
} >> "$EV/11_concurrencia_hilos.txt"

# 14 Rendimiento (pregunta 7)
encabezado 14_costo_rendimiento.txt "Evidencia adicional: costo del Broker
en rendimiento (pregunta 7)" \
"La misma operacion se envia 100 veces directo al Sistema 1 (puerto 6000,
publicado por Docker solo
para esta medicion) y 100 veces a traves del Broker (puerto 5000). La
diferencia es el costo del salto
adicional: una conexion TCP mas, marshalling/unmarshalling extra y la
busqueda en el registro."
$JAVA -cp "$CP" "$EV/EvidenciaRendimiento.java" 100 6000 5000 2>&1 | grep
-v '^WARNING' >> "$EV/14_costo_rendimiento.txt"

echo "parte 1 terminada"
```

parte2.sh

Genera las evidencias 08, 09 y 12; deja la consola del segundo proveedor en `proveedor2.log`.

```
#!/usr/bin/env bash
# Parte 2: evidencias que detienen componentes (07, 08, 09 y 12).
# Al final deja el stack como estaba: MariaDB, Broker y Sistema 1 arriba y
registrados.
source "$(dirname "$0")/comun.sh"

esperar_sano() {  # esperar_sano <contenedor>
  sleep 5  # tras docker start el estado tarda en pasar de "healthy"
(anterior) a "starting"
  for _ in $(seq 1 60); do
    [ "$(docker inspect -f '{{.State.Health.Status}}' "$1" 2>/dev/null)" =
"healthy" ] && return 0
    sleep 1
  done
  echo "TIMEOUT esperando a $1" >&2; return 1
}

esperar_linea() {  # esperar_linea <contenedor> <texto> <cantidad minima>
  for _ in $(seq 1 60); do
    [ "$(docker logs "$1" 2>&1 | grep -cF -- "$2")" -ge "$3" ] && return 0
    sleep 1
  done
  echo "TIMEOUT esperando '$2' en $1" >&2; return 1
}

detener_puerto() {  # detener_puerto <puerto>: detiene el proceso que
escucha en ese puerto del equipo
  powershell -NoProfile -Command "Get-NetTCPConnection -LocalPort $1 -State
Listen -ErrorAction SilentlyContinue | ForEach-Object { Stop-Process -Id
$_.OwningProcess -Force }"
}

#
------------------------------------------------------------------------
------------------
# 07 Excepcion producida por un proveedor
encabezado 07_excepcion_proveedor.txt "Prueba 7: Excepcion producida por un
proveedor" \
"Se detiene MariaDB mientras el Sistema 1 sigue arriba. Al atender la
solicitud, el proveedor lanza una
excepcion de acceso a datos; su despachador la convierte en una respuesta
ERROR_INTERNO y el Broker la
devuelve al cliente sin caerse. Al volver la base, la misma operacion
funciona sin reiniciar nada."
```

```
echo "## Paso 1: docker stop uady-mariadb" >> "$EV/
07_excepcion_proveedor.txt"
docker stop uady-mariadb >/dev/null
paso 07_excepcion_proveedor.txt "Paso 2: solicitud con la base detenida"
ev-07a inventario listarInventario '{}'
paso 07_excepcion_proveedor.txt "Paso 3: el Broker sigue atendiendo otras
solicitudes" ev-07b servicioFantasma listar '{}'
echo "## Paso 4: docker start uady-mariadb" >> "$EV/
07_excepcion_proveedor.txt"
docker start uady-mariadb >/dev/null && esperar_sano uady-mariadb
sleep 5  # margen para que MariaDB acepte conexiones de la red de Docker
paso 07_excepcion_proveedor.txt "Paso 5: la misma solicitud con la base de
vuelta (sin reiniciar el Broker ni el Sistema 1)" \
  ev-07c inventario listarInventario '{}'

#
---------------------------------------------------------------------------
-----------------
# 08 Cambio de proveedor sin modificar el cliente
encabezado 08_cambio_proveedor.txt "Prueba 8: Cambio de proveedor sin
modificar el cliente" \
"Un segundo proveedor (otra instancia del Sistema 1, fuera de Docker, en el
puerto 6001) se registra
en el Broker para los mismos servicios y sustituye al original; despues se
apaga el original. El cliente
ejecuta exactamente el mismo comando antes y despues: solo cambia el
proveedor que el Broker localiza."
paso 08_cambio_proveedor.txt "Paso 1: proveedor original" ev-08a
gestorTiendas listarTiendas '{}'

echo "## Paso 2: arranca el proveedor 2 (host.docker.internal:6001) y se
registra" >> "$EV/08_cambio_proveedor.txt"
set -a; source <(grep -E '^DB_(NAME|USER|PASSWORD)=' "$REPO/.env"); set +a
DB_HOST=localhost DB_PORT=3306 BROKER_HOST=localhost BROKER_PORT=5000 \
SERVICE_HOST=host.docker.internal SERVICE_PORT=6001 QR_DIR="${TMPDIR:-/
tmp}/qr-proveedor2" \
  $JAVA -Djava.awt.headless=true -jar "$REPO/sistema-centralizado/target/
sistema-centralizado.jar" \
  > "$EV/proveedor2.log" 2>&1 &
esperar_linea uady-broker "-> host.docker.internal:6001" 3
{
  grep -E "Proveedor registrado" "$EV/proveedor2.log"
  docker logs uady-broker 2>&1 | grep -F "[REGISTRO]" | grep -F
"host.docker.internal:6001"
  echo
```

```
} >> "$EV/08_cambio_proveedor.txt"

echo "## Paso 3: se apaga el proveedor original (docker stop uady-sistema-
centralizado)" >> "$EV/08_cambio_proveedor.txt"
docker stop uady-sistema-centralizado >/dev/null
echo >> "$EV/08_cambio_proveedor.txt"
paso 08_cambio_proveedor.txt "Paso 4: el mismo cliente, sin cambios, ahora
lo atiende el proveedor 2" ev-08b gestorTiendas listarTiendas '{}'

#
--------------------------------------------------------------------------
------------------
# 09 Proveedor caido: timeouts, reintentos y marca de no disponible
encabezado 09_proveedor_caido_timeout.txt "Evidencia adicional: proveedor
caido, reintentos limitados y desconexion" \
"Se apaga el proveedor activo (el proveedor 2) sin que se de de baja. El
Broker reintenta de forma
limitada (3 intentos, 200 ms entre ellos), responde con error y, tras 3
solicitudes fallidas, marca el
servicio como NO DISPONIBLE: la siguiente solicitud se rechaza de inmediato
sin esperar. Al volver un
proveedor y registrarse, el servicio se reactiva."
detener_puerto 6001
echo "## Paso 1: se detiene el proveedor 2 (puerto 6001)" >> "$EV/
09_proveedor_caido_timeout.txt"
echo >> "$EV/09_proveedor_caido_timeout.txt"
paso 09_proveedor_caido_timeout.txt "Paso 2: primera solicitud (3
intentos)" ev-09a inventario listarInventario '{}'
paso 09_proveedor_caido_timeout.txt "Paso 3: segunda solicitud" ev-09b
inventario listarInventario '{}'
paso 09_proveedor_caido_timeout.txt "Paso 4: tercera solicitud (alcanza el
umbral)" ev-09c inventario listarInventario '{}'
{
  echo "Marca de no disponible en la bitacora:"
  docker logs uady-broker 2>&1 | grep -F "NO DISPONIBLE"
  echo
} >> "$EV/09_proveedor_caido_timeout.txt"
paso 09_proveedor_caido_timeout.txt "Paso 5: cuarta solicitud (rechazo
inmediato, sin contactar al proveedor)" ev-09d inventario listarInventario
'{}'

echo "## Paso 6: vuelve el proveedor original (docker start uady-sistema-
centralizado) y se registra de nuevo" >> "$EV/
09_proveedor_caido_timeout.txt"
antes=$(docker logs uady-broker 2>&1 | grep -cF "registrar inventario ->
```

```
sistema-centralizado:6000")
docker start uady-sistema-centralizado >/dev/null
esperar_linea uady-broker "registrar inventario -> sistema-
centralizado:6000" $((antes + 1))
sleep 1
docker logs uady-broker 2>&1 | grep -F "[REGISTRO]" | tail -3 >> "$EV/
09_proveedor_caido_timeout.txt"
echo >> "$EV/09_proveedor_caido_timeout.txt"
paso 09_proveedor_caido_timeout.txt "Paso 7: el servicio vuelve a
responder" ev-09e inventario listarInventario '{}'

#
--------------------------------------------------------------------------
-----------------
# 12 El Broker deja de funcionar (preguntas 3 y 4)
encabezado 12_broker_caido.txt "Evidencia adicional: el Broker deja de
funcionar (preguntas 3 y 4)" \
"Se detiene el Broker. El cliente no puede comunicarse con ningun servicio
aunque el Sistema 1 siga
funcionando: el Broker es un punto unico de falla. Al reiniciarlo, su
registro (en memoria) esta vacio y
el Sistema 1 no se vuelve a registrar por si solo, asi que los servicios
aparecen como desconocidos hasta
reiniciar el proveedor. Esta limitacion es una mejora pendiente (re-
registro periodico, error #2)."
echo "## Paso 1: docker stop uady-broker" >> "$EV/12_broker_caido.txt"
docker stop uady-broker >/dev/null
paso 12_broker_caido.txt "Paso 2: el cliente intenta una solicitud" ev-12a
gestorTiendas listarTiendas '{}'
{
  echo "## Paso 3: el Sistema 1 sigue en ejecucion (docker ps)"
  docker ps --filter name=uady- --format "{{.Names}} | {{.Status}}"
  echo
  echo "## Paso 4: docker start uady-broker"
} >> "$EV/12_broker_caido.txt"
docker start uady-broker >/dev/null && esperar_sano uady-broker
paso 12_broker_caido.txt "Paso 5: el Broker volvio, pero su registro esta
vacio" ev-12b gestorTiendas listarTiendas '{}'
echo "## Paso 6: se reinicia el Sistema 1, que se registra al arrancar" >>
"$EV/12_broker_caido.txt"
antes=$(docker logs uady-broker 2>&1 | grep -cF "registrar gestorTiendas ->
sistema-centralizado:6000")
docker restart uady-sistema-centralizado >/dev/null
esperar_linea uady-broker "registrar gestorTiendas -> sistema-
centralizado:6000" $((antes + 1))
```

```
sleep 1
echo >> "$EV/12_broker_caido.txt"
paso 12_broker_caido.txt "Paso 7: el servicio vuelve a responder" ev-12c
gestorTiendas listarTiendas '{}'

echo "parte 2 terminada"
docker ps --filter name=uady- --format "{{.Names}} | {{.Status}}"
```

parte3.sh

Genera la evidencia 07 (excepción del proveedor y base de datos detenida).

```
#!/usr/bin/env bash
# Parte 3: prueba 7 (excepcion producida por un proveedor), de forma
determinista.
source "$(dirname "$0")/comun.sh"

esperar_sano() {
  sleep 5
  for _ in $(seq 1 60); do
    [ "$(docker inspect -f '{{.State.Health.Status}}' "$1" 2>/dev/null)" =
"healthy" ] && return 0
    sleep 1
  done
  echo "TIMEOUT esperando a $1" >&2; return 1
}

A=07_excepcion_proveedor.txt
encabezado $A "Prueba 7: Excepcion producida por un proveedor" \
"Caso A: se provoca una excepcion real dentro del proveedor (la tabla
'tiendas_almacenes' deja de existir de forma
temporal, asi que la consulta JDBC lanza SQLException). El despachador del
Sistema 1 convierte la
excepcion en una respuesta ERROR_INTERNO, conservando el id; el Broker la
devuelve al cliente y sigue
atendiendo. Al restaurar la tabla, la misma operacion funciona sin
reiniciar el Broker ni el Sistema 1.

Caso B (complemento): si la base de datos se detiene por completo, el
proveedor no responde dentro del
timeout de lectura del Broker (5 s); el Broker reintenta 3 veces y responde
TIMEOUT."
```

```
{
  echo "## Caso A, paso 1: RENAME TABLE tiendas_almacenes TO
tiendas_respaldo"
  sql "RENAME TABLE tiendas_almacenes TO tiendas_respaldo"
  echo
} >> "$EV/$A"
paso $A "Caso A, paso 2: solicitud mientras la tabla no existe" ev-07-c1
gestorTiendas listarTiendas '{}'
{
  echo "Consola del Sistema 1 en ese momento (docker logs uady-sistema-
centralizado):"
  docker logs uady-sistema-centralizado 2>&1 | grep -iE "tiendas|SQL" |
tail -3
  echo
} >> "$EV/$A"
paso $A "Caso A, paso 3: el Broker sigue atendiendo otras operaciones"
ev-07-c2 inventario listarInventario '{}'
{
  echo "## Caso A, paso 4: RENAME TABLE tiendas_respaldo TO
tiendas_almacenes"
  sql "RENAME TABLE tiendas_respaldo TO tiendas_almacenes"
  echo
} >> "$EV/$A"
paso $A "Caso A, paso 5: la misma solicitud, sin reiniciar nada" ev-07-c3
gestorTiendas listarTiendas '{}'

echo "## Caso B, paso 1: docker stop uady-mariadb" >> "$EV/$A"
docker stop uady-mariadb >/dev/null
{
  echo "## Caso B, paso 2: solicitud con la base detenida (cliente con
espera de 30 s para ver la respuesta del Broker)"
  echo
  $JAVA -DtimeoutMs=30000 -cp "$CP" "$EV/EvidenciaCliente.java" ev-07-d1
inventario listarInventario '{}' 2>&1 | grep -v '^WARNING'
  echo
  echo "Bitacora del Broker (docker logs uady-broker):"
  bitacora ev-07-d1
  echo
  echo "Observacion: el Broker tarda unos 15 s (3 intentos x 5 s) en
responder TIMEOUT. El cliente del"
  echo "Sistema 2 espera 5 s, asi que el usuario veria primero su propio
timeout. Es la limitacion #1 de la"
  echo "revision de errores (alinear el timeout del cliente con el peor
caso del Broker y no reintentar"
  echo "operaciones que ya llegaron al proveedor)."
```

```
  echo
  echo "## Caso B, paso 3: docker start uady-mariadb"
} >> "$EV/$A"
docker start uady-mariadb >/dev/null && esperar_sano uady-mariadb
sleep 5
paso $A "Caso B, paso 4: con la base de vuelta, sin reiniciar nada" ev-07-
d2 inventario listarInventario '{}'

echo "parte 3 terminada"
```